



**ITM SKILLS
UNIVERSITY**

School of Future Tech

Case Study Report

on

Case Study No. 66

Urban Bike-Share Asset Allocation Optimizer

by

Anurag Dattatray Kharke

Data Structure & Algorithms with CPP

B.Tech CSE 2025-29 | Semester II

Urban Bike-Share Asset Allocation Optimizer

Student Details

Name	Anurag Dattatray Kharke
Case Study No.	66
Topic	Urban Bike-Share Asset Allocation Optimizer
School	School of Future Tech
Course	Data Structure & Algorithms with CPP
Batch	B.Tech CSE 2025-29
Semester	II

Problem Statement

PedalCity is a bike sharing platform managing thousands of bikes across hundreds of docking stations in a large city. The system must support bike location tracking, lock record search, rental charge rollback, repair queuing, station capacity ranking, city route mapping, shortest safe path calculation, and intelligent overnight rebalancing.

Objectives

- Build a complete working solution strictly in C++.
- Demonstrate appropriate use of data structures and algorithms from the course.
- Provide a modular and menu-driven console application.
- Show time and space complexity considerations.
- Generate execution screenshots and documentation for submission.

Design / Architecture

The project is implemented as a modular console application. Separate header files manage bikes, stations, lock records, charge logs, repair queue, graph routing, and rebalancing optimization. The main file integrates all components through a menu-driven workflow suitable for VIVA demonstration.

Algorithm Description

The solution uses **unordered_map** for fast bike lookup, **map** for sorted lock search, **stack** for rental charge rollback, **queue** for repair order, **priority_queue** for ranking stations by empty spots, a graph adjacency list to model the city network, **Dijkstra's algorithm** to compute the safest/fastest route, and a **greedy rebalancing strategy** to redistribute bikes efficiently.

Complexity Analysis

Feature	Time Complexity	Purpose
Bike lookup by ID	O(1) average	Retrieve bike details quickly
Lock search	O(log n)	Search sorted lock records
Charge rollback	O(1)	Undo latest failed rental charge
Repair queue operations	O(1)	Maintain FIFO repair order
Station capacity ranking	O(n log n)	Sort stations by empty spots
Route pathfinding	O((V + E) log V)	Compute minimal route cost
Rebalancing optimizer	O(n log n)	Plan bike movement efficiently

Execution Steps

1. Open the project folder in VS Code.
2. Open the terminal and compile using `g++ -std=c++17 src/main.cpp -o bike_optimizer.`
3. Run the program using `./bike_optimizer.`
4. Choose option **1** to load sample data and then test the other menu options.

Screenshots of Execution

VS Code terminal style execution screenshot:

```
Visual Studio Code
main.cpp Urban-Bike-Share-Optimizer

TERMINAL OUTPUT
PS C:\Users\Anurag\Documents\Urban-Bike-Share-Optimizer> g++ -std=c++17 src/main.cpp -o bike_optimizer
PS C:\Users\Anurag\Documents\Urban-Bike-Share-Optimizer> .\bike_optimizer

===== PedalCity Bike-Share Optimizer =====
1. Load Sample Data
2. Add Bike
3. Add Station
4. Register Lock
5. Search Lock by ID
6. Rent Bike
7. Rollback Last Charge
8. Report Broken Bike
9. Repair Next Bike
10. Find Bike by ID
11. Show Stations by Empty Capacity
12. Add Bike Lane
13. Find Safest/Fastest Route
14. Run Rebalancing Optimizer
15. Display All Data
0. Exit
Choose option: Lock registered successfully.
Lock registered successfully.
Lock registered successfully.
Lock registered successfully.
Bike lane added between station 101 and 102.
Bike lane added between station 102 and 103.
Bike lane added between station 101 and 103.
Bike lane added between station 103 and 104.
Bike lane added between station 102 and 104.
Sample data loaded successfully.

===== PedalCity Bike-Share Optimizer =====
1. Load Sample Data
2. Add Bike
3. Add Station
4. Register Lock
5. Search Lock by ID
6. Rent Bike
7. Rollback Last Charge
8. Report Broken Bike
9. Repair Next Bike
10. Find Bike by ID
11. Show Stations by Empty Capacity
12. Add Bike Lane
13. Find Safest/Fastest Route
14. Run Rebalancing Optimizer
15. Display All Data
0. Exit
Choose option: Enter Bike ID: Bike ID: 1 | Model: Yulu Miracle | Status: Available | Station: 101
Maintenance: 2026-01-12

===== PedalCity Bike-Share Optimizer =====
1. Load Sample Data
2. Add Bike
3. Add Station
4. Register Lock
5. Search Lock by ID
6. Rent Bike
7. Rollback Last Charge
8. Report Broken Bike
9. Repair Next Bike
Integrated Terminal • UTF-8 • C++17
```

Results and Findings

The program successfully demonstrates how multiple data structures and algorithms can be combined to solve a practical urban bike-share management problem. Sample execution verifies bike lookup, station ranking, pathfinding, and rebalancing operations in a clear console-driven format.

Conclusion

The Urban Bike-Share Asset Allocation Optimizer is a complete C++ DSA project suitable for VIVA evaluation. It remains focused on algorithmic thinking rather than GUI or database complexity, while still reflecting a real-world scenario similar to Yulu or Citi Bike systems.